TYBCA · SEMESTER COURSE

Introduction to Machine Learning using Python

Concepts, process, and the essential Python toolkit for data science.

MIT WPU, Kothrud Department of Computer Science and Applications

SESSION OVERVIEW

Agenda

01 Machine learning vs traditional programming

02 Steps of the machine learning process

03 Types of machine learning

04 Applications of machine learning

05 Python libraries for data science

06 NumPy · Pandas · SciPy · Matplotlib

TOPIC 01

Machine Learning vs Traditional Programming

Traditional programming	Data + Rules	→	Computer	→	Output
Machine learning	Data + Output	→	Computer	→	Rules (Model)

Example — a spam filter: instead of hand-coding rules for every spam word, we supply thousands of labelled emails and the algorithm learns the classification rules on its own.

A Side-by-Side Comparison

ASPECT	Traditional programming	Machine learning
Approach	Programmer writes explicit rules	Algorithm learns rules from data
Logic	Deterministic, fixed at design time	Statistical, inferred from patterns
Adaptability	Must be rewritten when rules change	Retrained on new data, improves over time
Data dependence	Works with little or no data	Requires large, quality datasets
Examples	Calculator, payroll system	Spam filter, recommendations, face recognition

TOPIC 02

Steps of the Machine Learning Process

1
Data collection
Gather raw data from databases, sensors, or the web

2
Data preparation
Clean, normalise, and split into training / test sets

3
Model selection
Choose an algorithm suited to the problem type

4
Training and Testing
Feed training data so the model learns its parameters

5
Evaluation
Measure accuracy on unseen test data

6
Tuning
Adjust hyperparameters to improve performance

7
Prediction
Deploy the model to make decisions on new data

Note

Steps 1–2 typically take 60–80% of project time

TOPIC 03

Types of Machine Learning

Supervised

Learns from labelled examples — every input has a known output

Feedback: correct answers provided

Unsupervised

Finds hidden structure in unlabelled data on its own

Feedback: none — patterns only

Semi-supervised

Combines a small labelled set with a large unlabelled one

Feedback: partial labels

Reinforcement

An agent learns by acting in an environment, by trial and error

Feedback: rewards and penalties

Supervised Learning

The model learns from labelled data — each training example pairs an input with its known output, like learning with a teacher who corrects mistakes.

- **Classification** — predicts a category: spam / not spam
- **Regression** — predicts a number: house price, temperature
- **Algorithms** — linear regression, decision trees, k-NN, SVM

EXAMPLE · HOUSE PRICE PREDICTION

Input: area, bedrooms, location

Label: actual sale price

Learned: price = f(area, bedrooms, ...)

Use: predict the price of a new listing

Unsupervised Learning

The model receives unlabelled data and must discover hidden patterns, groupings, or structure on its own — no teacher, no correct answers.

- **Clustering** — grouping similar records: k-means, hierarchical
- **Dimensionality reduction** — compressing features: PCA
- **Association rules** — market-basket analysis: Apriori

EXAMPLE · CUSTOMER SEGMENTATION

Input: purchase history, no labels

Found: 3 natural groups of shoppers

Use: targeted offers per segment

Semi-Supervised Learning

A middle path: train with a small labelled dataset plus a large unlabelled one. Useful because labelling data is slow and expensive, while raw data is abundant.

- **Why** — expert labelling (doctors, linguists) is costly
- **How** — learn from labels, refine with pseudo-labelling
- **Where** — medical imaging, speech, web classification

EXAMPLE · MEDICAL SCANS

Labelled: 500 scans marked by radiologists

Unlabelled: 50,000 raw scans

Result: near-supervised accuracy at a fraction of the labelling cost

Reinforcement Learning

An agent learns by trial and error: it acts in an environment, receives rewards or penalties, and gradually learns a policy that maximises total reward.

- **Vocabulary** — agent, environment, state, action, reward, policy
- **Analogy** — training a dog with treats and corrections
- **Uses** — game playing (AlphaGo), robotics, self-driving

THE FEEDBACK LOOP

Agent takes an action

↓

Environment returns new state + reward

↓

Policy is updated — repeat

TOPIC 04

Applications of Machine Learning

SUPERVISED

Spam filtering · credit-risk scoring · medical diagnosis · weather forecasting

UNSUPERVISED

Customer segmentation · news topic grouping · fraud & anomaly detection

SEMI-SUPERVISED

Photo tagging · speech recognition · web page classification

REINFORCEMENT

Game AI (AlphaGo) · robotics · self-driving cars · recommendation ranking

Everyday touchpoints: UPI fraud alerts, OTT recommendations, voice assistants, maps ETA prediction.

Key Takeaways — Machine Learning Concepts

1. ML inverts traditional programming: it learns rules from data instead of executing hand-written rules.
2. Every ML project follows the same pipeline — collect, prepare, select, train, evaluate, tune, predict.
3. The four types differ by feedback: labels (supervised), none (unsupervised), partial (semi-supervised), rewards (reinforcement).
4. Choosing the right type starts with one question: what kind of data and feedback do we have?

TOPIC 05

Python Libraries for Data Science

Python dominates data science because of its readable syntax and a mature, free, open-source ecosystem built by a large community.

- Simple syntax — more time on the problem, less on the code
- Rich ecosystem — a library for every step of the pipeline
- Interoperable — libraries share NumPy arrays as a common format

Matplotlib	visualisation
Pandas	data analysis
SciPy	scientific computing
NumPy	the foundation — fast arrays

NumPy – Numerical Computing

- **ndarray** — fast, fixed-type, n-dimensional array
- **Vectorised** — array maths runs in compiled C, 10–100× faster than loops
- **Broadcasting** — apply operations across whole arrays at once
- **Foundation** — Pandas, SciPy, and scikit-learn build on it

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr * 2)          # [ 2  4  6  8 10]
print(arr.mean())       # 3.0

m = np.zeros((2, 3))    # 2×3 matrix of 0s
print(m.shape)          # (2, 3)
```

Pandas – Data Analysis

- **Series** — one-dimensional labelled array
- **DataFrame** — 2-D table with named columns, like a spreadsheet
- **I/O** — read and write CSV, Excel, SQL, JSON
- **Cleaning** — filtering, grouping, missing-value handling

```
import pandas as pd

df = pd.read_csv("students.csv")
df.head()               # first 5 rows
df.describe()           # summary stats

toppers = df[df["marks"] > 75]
df = df.fillna(0)       # fix missing values
```

SciPy – Scientific Computing

- **Built on NumPy** — adds ready-made scientific algorithms
- **scipy.stats** — distributions, t-tests, correlation
- **scipy.optimize** — minimisation, curve fitting
- **scipy.linalg / integrate / signal** — linear algebra, integration, signals

```
from scipy import stats

a = [72, 75, 78, 80, 82]
b = [65, 68, 70, 72, 74]

t, p = stats.ttest_ind(a, b)
print(p)   # is the difference
           # statistically significant?
```

Matplotlib — Data Visualisation

- **pyplot** — the standard plotting interface
- **Line** — trends over time · **Bar** — comparisons
- **Scatter** — relationships · **Histogram** — distributions
- **Always label** — title, xlabel, ylabel on every chart

```
import matplotlib.pyplot as plt

marks = [65, 72, 78, 85, 90]
sems  = [1, 2, 3, 4, 5]

plt.plot(sems, marks, marker="o")
plt.title("Marks per Semester")
plt.xlabel("Semester")
plt.ylabel("Marks")
plt.show()
```

Choosing the Right Library

LIBRARY	USE IT FOR	CORE OBJECT / INTERFACE
NumPy	Fast numeric arrays, matrix maths	`ndarray`
Pandas	Tabular data, cleaning, analysis	`DataFrame`, `Series`
SciPy	Statistics, optimisation, integration	`scipy.stats`, `scipy.optimize`
Matplotlib	Charts: line, bar, scatter, histogram	`pyplot`

Complementary, not competing — a typical data-science script imports all four.

Key Takeaways — Python Libraries & Tools

1. The stack is layered: NumPy arrays underpin SciPy, Pandas, and everything above them.
2. Vectorised NumPy operations replace slow Python loops — think in arrays, not elements.
3. Pandas is where data preparation happens — read, inspect, filter, and fix missing values.
4. SciPy supplies textbook maths ready-made; Matplotlib turns every result into a labelled chart.

QUICK QUIZ

Review Questions

Q1 How does machine learning differ from traditional programming in where the rules come from?

Q2 List the seven steps of the machine learning process in order.

Q3 What distinguishes supervised from unsupervised learning?

Q4 Which ML type does AlphaGo use, and what feedback does it learn from?

Q5 Which library and data structure would you use to load and filter a CSV file?

Q6 Name two Pandas methods for handling missing values.

FURTHER STUDY

References & Further Reading

Book	Géron, A. — *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, O'Reilly
Book	Müller, A. & Guido, S. — *Introduction to Machine Learning with Python*, O'Reilly
Book	McKinney, W. — *Python for Data Analysis*, O'Reilly (by the creator of Pandas)
Docs	numpy.org · pandas.pydata.org · scipy.org · matplotlib.org — official tutorials, free
Course	Andrew Ng — *Machine Learning Specialization*, Coursera (audit free)

END OF SESSION

Thank You

Next session: building your first supervised model with scikit-learn. Try the four code snippets in lab before then.